



Sandya Mannarswamy

CODE SPORT

In this month's column, we take a quick look at the threading support introduced in the new standard for C++11.

One of the major events of 2011 in the programming world was the official release of the new C++11 standard. This was not a minor revamp with just a few new features, but a full-blown release of the new standard for C++, with a number of radically new features, such as lambda expressions, automatic type deduction of objects, rvalue references, etc. As Bjarne Stroustrup put it aptly, "C++11 feels like a new language—the pieces just fit together better." Other than the language features, there are a whole new set of features introduced in the C++ standard library as well. A number of new container classes, new algorithms, smart pointers, atomic operations and a new type of traits have been introduced. However, the most important addition to the C++ standard library is the threading support. In this column, we will discuss how threads are supported in C++11, with a couple of examples. First, let's start off with concurrency, and how threads can be used to implement it.

Concurrency and threads

Concurrency refers to the ability to perform multiple independent sets of operations in parallel, instead of executing the operations in a single stream, sequentially. Concurrency can be achieved by means of either processes or threads. When concurrency is supported by means of a multi-process mechanism, each process gets its own copy of the data, and it needs to communicate using standard inter-process communication mechanisms such as pipes, message-passing or explicitly

created shared memory segments. The other way of achieving concurrency is through multiple threads in a single process. Using multiple threads of the same process to achieve concurrency is a lightweight approach, since the data is shared across threads of the same process, using shared memory. However, this requires synchronisation and cooperation among the threads, and hence imposes a certain programming burden on the developer.

Mechanisms for creating threads

In the UNIX/Linux world, the standard mechanism of creating threads is through the POSIX threads (popularly known as *pthread*s) interfaces (Windows supports *WinThreads*, which offers similar functionality). Here is a simple program that prints 'hello world' with just two threads:

```
#include <pthread>
void print_hello(void)
{
    printf("hello world\n");
}

int main (void)
{
    printf("Main Thread - hello world\n");
    pthread_t thread_secondary;
    //create the auxiliary thread
    pthread_create(&thread_secondary, NULL,
    &print_hello, NULL);
    //join the auxiliary thread with main thread
    pthread_join(&thread_secondary, NULL);
    return 0;
}
```